



南开大学
Nankai University

南 开 大 学

计 算 机 学 院

并行程序设计实验报告

GPU 编程学习与矩阵 SVD 选题 GPU 实验

姓名：邹俊轩

学号：2412592

年级：2024 级

专业：计算机科学与技术

指导教师：王刚

代码仓库：[GITHUB.COM/POINTZU/GPU-SVD-CUDA-EXPERIMENT](https://github.com/POINTZU/GPU-SVD-CUDA-EXPERIMENT)

2026 年 6 月 20 日

摘要

本报告围绕并行程序设计课程 GPU 编程实验与矩阵 SVD 选题展开。前半部分整理 AMD HIP 六个 notebook 中主要需要体现学习通过的代码填空、运行结果和性能分析，重点关注线程层次、访存合并、共享内存、原子操作、规约、Monte Carlo 积分和 K-Means 等 GPU 编程模式。后半部分针对 SVD 选题，说明 `bidiagonalization.cpp` 中 Householder 上二对角化阶段的 GPU 化实现思路，并结合服务器已有运行数据分析正确性、耗时分布。

关键词：GPU 编程；AMD HIP；SVD；Householder；CUDA/HIP；GEMV；GER；profiling

目录

1 实验概述	1
1.1 实验目标	1
2 AMD HIP Notebook 学习内容	1
2.1 Lab1: Vector Add 与线程层次	1
2.2 Lab2: Matrix Transpose 与访存合并	2
2.3 Lab3: Histogram 与原子竞争	3
2.4 Lab4: Reduction 与 occupancy 分析	4
2.5 Lab5: Monte Carlo Integration	6
2.6 Lab6: K-Means	7
3 SVD GPU 实验	8
3.1 计算流程	8
3.2 Householder 阶段的 GPU 化	8
3.3 GKH 阶段说明	8
4 服务器运行与结果分析	12
4.1 服务器与本地运行结果	12
4.2 本地 NVIDIA GPU 测试	12
4.3 本地 nvcc CUDA C++ SVD 运行	13
4.4 VTune 与 AMD uProf profiling	15
4.5 结果分析	16
5 总结	16
6 SVD 选题 AI 问答记录	17
7 截图与材料清单	20
8 AMD Notebook 补充截图附录	20

1 实验概述

1.1 实验目标

本次实验包含两条主线。第一条主线是 AMD HIP notebook 学习实验，目标是补全 GPU 程序中的关键代码，并通过截图和性能数据说明不同并行模式的效果。第二条主线是 SVD 选题，目标是在不修改 `main.cpp/test.sh/qsub.sh` 的前提下，关注 `bidiagonalization.cpp` 中的上二对角化阶段，将其中 GEMV/GER 型 Householder 更新搬到 GPU 上，并分析加速效果和额外开销。

2 AMD HIP Notebook 学习内容

2.1 Lab1: Vector Add 与线程层次

Vector Add 的核心填空是全局线程编号计算：

Vector Add kernel

```
1 int idx = blockIdx.x * blockDim.x + threadIdx.x;
2 if (idx < N) {
3     C[idx] = A[idx] + B[idx];
4 }
```

对于 $N = 1000$ ，若线程总数为 1024，则线程利用率为

$$\frac{1000}{1024} = 97.66\%.$$

相同线程利用率不必然对应相同性能，因为实际运行还会受到 wavefront 对齐、occupancy、调度粒度和 kernel launch 固定开销影响。

Question: Which block size gives the best thread utilization? Why might this not always be the best choice?

Your answer: All four block sizes have the same thread utilization here: $1000 / 1024 = 97.66\%$. This does not always make them equally good, because real performance also depends on wavefront alignment, occupancy, scheduling overhead, register/shared-memory pressure, and memory bandwidth behavior. A larger block can reduce the number of blocks but may reduce flexibility; a smaller block can increase scheduling overhead or waste lanes if it is not wavefront-aligned.

图 1: Lab1 GPU 检测与运行环境输出

Exercise 4: Analyze Your Results

Based on the multi-trial results above:

- Which block size achieved the **lowest mean execution time**? **Block size 512**, with mean execution time **0.1382 ms**.
- Which block size achieved the **highest memory bandwidth**? **Block size 512**. For fixed N , achieved bandwidth is inversely proportional to kernel time, so the lowest mean time gives the highest bandwidth.
- Are the differences between block sizes **statistically significant** (look at error bars)? **Mostly not strongly significant**. The mean times are all close, roughly from **0.1382 ms to 0.1451 ms**, and several standard deviations/error bars overlap. Block size 512 is the best in this run, but the advantage is small.
- Why does Vector Add performance depend more on bandwidth than on block size?

Your answer: Vector addition has very low arithmetic intensity: each element needs only one floating-point addition, but it requires reading $A[i]$, reading $B[i]$, and writing $C[i]$ (about 12 bytes per element). Because the computation per byte is tiny, the kernel is mainly limited by global memory bandwidth. Once the block size is wavefront-aligned and provides enough parallelism to keep the GPU busy, changing block size has a smaller effect than the memory subsystem. In the measured results, all block sizes from 16 to 1024 are close, while 512 gives the best observed time and bandwidth.

Measured summary:

Block Size	Mean (ms)	Std (ms)	Min (ms)	Max (ms)	Trials
16	0.1424	0.0030	0.1388	0.1494	10
32	0.1451	0.0061	0.1388	0.1538	10
64	0.1430	0.0075	0.1292	0.1503	10
128	0.1421	0.0046	0.1386	0.1497	10
256	0.1391	0.0067	0.1295	0.1499	10
512	0.1382	0.0032	0.1300	0.1412	10
1024	0.1413	0.0017	0.1397	0.1444	10

图 2: Lab1 block size benchmark 运行结果

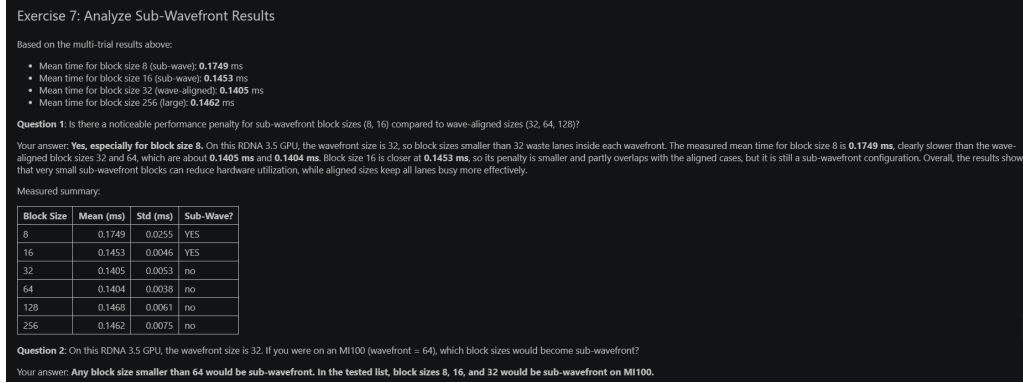


图 3: Lab1 sub-wavefront 对齐影响实验结果

从截图可见，程序能够正确检测设备并完成不同 block size 的 benchmark。sub-wavefront 实验说明，过小的 block 会浪费 wavefront lane；在 AMD GPU 默认 wave32 环境下，线程块大小应尽量与 wavefront 的整数倍对齐。

2.2 Lab2: Matrix Transpose 与访存合并

矩阵转置中输出矩阵尺寸为 cols x rows，因此输出线性下标应使用输出行长度 rows：

Naive transpose write index

```
1 output[idx * rows + idy] = input[idy * cols + idx];
```

测试	维度	元素数	kernel 时间
Test 1	16 × 16	256	0.0077 ms
Test 2	128 × 32	4096	0.0108 ms
Test 3	1 × 1024	1024	0.0087 ms
Test 4	1001 × 2001	2003001	0.2352 ms

表 1: Matrix transpose kernel time



图 4: Lab2 转置测试样例运行输出

Exercise 5: Tiled Transpose Analysis

Question 1: Why do we add `+1` to the shared memory declaration (`tile[BLOCK_SIZE][BLOCK_SIZE + 1]`)?

Your answer: The `+1` padding changes the shared-memory row pitch from 32 floats to 33 floats. When the tile is accessed in transposed order, this prevents all threads from hitting the same shared-memory bank, avoiding severe bank conflicts.

Question 2: In the write step, why do we swap `blockIdx.x` and `blockIdx.y`?

Your answer: The input tile at block `(blockIdx.x, blockIdx.y)` becomes an output tile at `(blockIdx.y, blockIdx.x)` after transpose. Swapping the block coordinates makes each block write the transposed tile to the correct output position while keeping global writes coalesced.

Question 3: How much shared memory does each block use for `BLOCK_SIZE=32`?

Your calculation: $32 * (32 + 1) * \text{sizeof(float)} = 32 * 33 * 4 = 4224$ bytes.

图 5: Lab2 多轮 benchmark 统计输出

12. Challenge Exercises

Challenge 1: Implement Tiled Transpose

Modify `main.hip` to use the shared memory tiling approach from Section 9. Compare performance with the naive version.

Challenge 2: Bank Conflict Analysis

Test with and without the `+1` padding in shared memory. Measure the performance difference.

Challenge 3: In-Place Transpose

For square matrices, implement in-place transpose using only $O(1)$ extra memory.

Challenge 4: Rectangular Tile Sizes

Experiment with non-square tile sizes (e.g., 32×8 or 16×64). When might these be beneficial?

Challenge 5: Double-Buffering

Implement double-buffering to overlap memory transfers with computation for very large matrices.

图 6: Lab2 转置访存模式分析截图

naive transpose 能通过正确性测试，但写输出矩阵时是跨步访问，难以形成理想的 coalesced write。大规模矩阵耗时增加明显，但并不完全按元素数线性增长，说明小规模测试中固定 launch 和同步开销占比较高。

2.3 Lab3: Histogram 与原子竞争

Naive GPU histogram 的问题是每个 bin 对整段输入扫描一次，全局内存读次数为

$$N \times \text{num_bins}.$$

优化版本使用 shared memory histogram：每个 block 先在共享内存中统计局部结果，再将每个 bin 的局部结果用 global atomic 合并。

实现	Test 2	Test 4
Serial CPU	2.3624 ms	22.6050 ms
Naive GPU	23.2188 ms	2526.8180 ms
Optimized GPU	0.4661 ms	4.4517 ms

表 2: Histogram 性能结果

Exercise 1: Analyze the Problem

Question 1: How many times is each input element read from global memory?

Your answer: Each input element is read `num_bins` times from global memory, because every bin has one block that scans the whole input array.

Question 2: If `num_bins = 256` and `N = 10M`, what is the total global memory traffic?

Your calculation: $N * \text{num_bins} = 10,000,000 * 256 = 2,560,000,000$ global integer reads, which is about $2,560,000,000 * 4 \text{ B} = 10.24 \text{ GB}$ of input-read traffic.

Question 3: Why is this approach inefficient?

Your answer: It repeats the same full-array scan once per bin, so the work and global memory reads are $O(N * \text{num_bins})$ instead of $O(N)$. Most reads only check values that do not match the block's bin, so memory bandwidth is wasted and the kernel scales poorly as the number of bins grows.

图 7: Lab3 histogram 正确性与基础 benchmark 输出

Exercise 5: Contention Scenarios

Scenario 1: All input values are the same (e.g., all zeros)

- What happens to performance? Performance becomes the worst among these distributions; the optimized kernel slows down because many threads update the same histogram bin.
- Why? All threads repeatedly execute `atomicAdd(&data[0], 1)`, so the shared-memory atomic operations serialize on one address. The two-level design still avoids most global atomic contention, but the block-local shared-memory contention is maximal.

Scenario 2: Input values are uniformly distributed across all bins

- Expected contention level: Low to moderate.
- Why? Updates are spread across all bins, so fewer threads target the same shared-memory address at the same time. The atomic operations are distributed more evenly, which reduces serialization and improves throughput.

图 8: Lab3 optimized histogram 性能对比

10. Summary

Key Concepts

Concept	Description
Race Condition	Multiple threads modifying same memory without synchronization
Atomic Operation	Indivisible read-modify-write operation
Memory Contention	Performance loss when atomics conflict frequently
Privatization	Using local (shared) memory to reduce global contention

Optimization Strategy

1. **Privatize** the histogram in shared memory (one per block)
2. Use **local atomics** (faster, less contention)
3. **Merge** block-local results into global histogram
4. Minimize global atomic operations

When to Use Each Approach

Scenario	Recommended Approach
Few bins, large N	Shared memory privatization
Many bins (> 1024)	May need multi-pass or sorting-based
Highly skewed data	Consider warp-level privatization

图 9: Lab3 speedup 与 atomic contention 分析

截图中的 optimized GPU 明显快于 naive GPU。优化收益来自两个方面：一是避免对全局输入数组进行 bin 数量倍的重复扫描；二是把大量竞争限制在 block 内共享内存中，减少 global atomic 压力。

2.4 Lab4: Reduction 与 occupancy 分析

Tree reduction 对 256 个元素进行块内规约，需要

$$\log_2 256 = 8$$

步。若 $N = 1024$ 且 block size 为 256，则需要 $\lceil 1024/256 \rceil = 4$ 个 block。

Block Size	Warps/Block	Occupancy	Time
64	2	100.00%	0.0737 ms
128	4	100.00%	0.0664 ms
256	8	100.00%	0.0641 ms
512	16	100.00%	0.0694 ms
1024	32	100.00%	0.0668 ms

表 3: Reduction occupancy sweep

策略	时间	结果
Naive Atomic	0.0856 ms	-74993790
Tree Shared Memory	0.0628 ms	-74993790
Tree + Warp Shuffle	0.0588 ms	-74993790
Multi-Element + Tree + Shuffle	0.0478 ms	-74993790

表 4: Reduction 四种策略对比

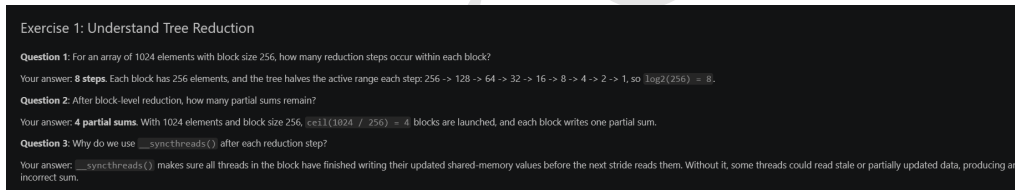


图 10: Lab4 reduction 基础运行输出

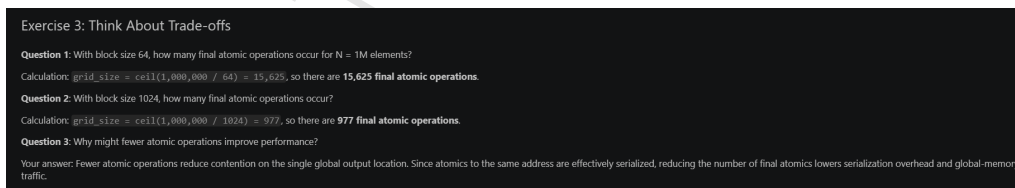


图 11: Lab4 occupancy sweep 结果

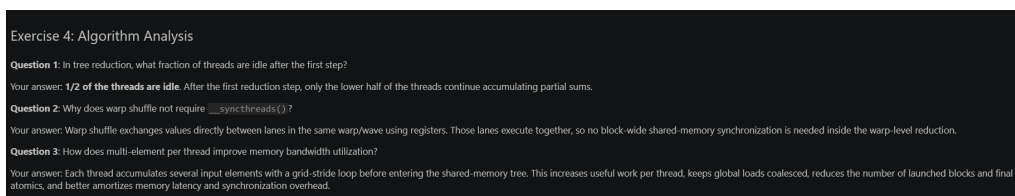


图 12: Lab4 多种 reduction 策略 benchmark

所有 block size 都可以达到较高 occupancy，但最佳性能并不一定出现在最大 block size。原因是 reduction 还受到同步次数、共享内存访问、warp 内通信和最终 atomic 次数影响。multi-

element + tree + shuffle 最快, 说明减少全局 atomic 并利用 warp shuffle 可以进一步降低开销。

2.5 Lab5: Monte Carlo Integration

Atomic 版本中, 每个线程把自己的样本贡献直接加到同一个全局结果:

Monte Carlo atomic accumulation

```
1 double tmp = (b - a) * y_samples[idx] / n_samples;
2 atomicAdd(result, tmp);
```

该写法总工作量为 $O(n)$, 但所有线程竞争同一个全局地址, global atomic accumulation 成为瓶颈。

Test 1 的手算结果为

$$\sum y_i = 23.7507, \quad \text{result} = \frac{2}{8} \times 23.7507 = 5.937675.$$

测试	样本数	时间
Test 4	100000	real 0m0.074s
Test 7	1000000	real 0m0.878s
Test 8	10000000	real 0m8.425s

表 5: Monte Carlo scaling

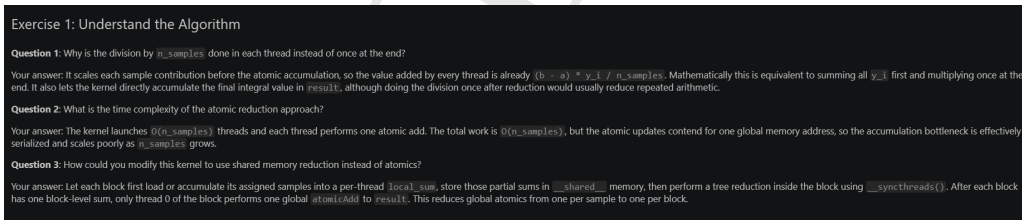


图 13: Lab5 Monte Carlo 小规模样例输出

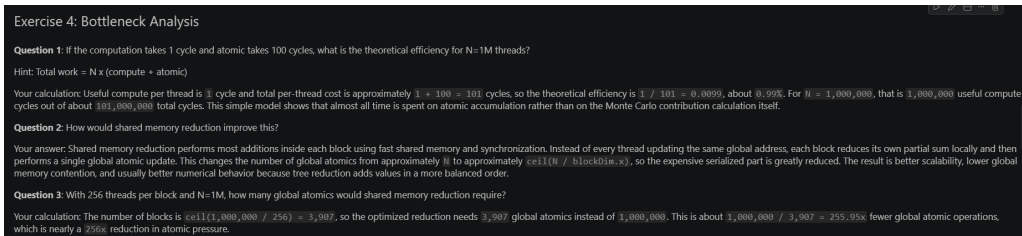


图 14: Lab5 Monte Carlo scaling 运行结果

Exercise 6: Optimization Analysis

Question 1: In the optimized version, how many global atomics occur for $N=1M$ with 256 blocks?

Your answer: 256 global atomics, because each block produces one partial sum and only thread 0 in each block performs one `atomicAdd`. Compared with the original one-atomic-per-sample kernel, this reduces the atomic count from 1,000,000 to 256, or about 3900x fewer global atomic operations.

Question 2: What is the purpose of the grid-stride loop `for (int i = idx; i < n_samples; i += blockDim.x * gridDim.x)`?

Your answer: The grid-stride loop lets a fixed number of launched threads cover any number of input samples. Each thread processes multiple elements separated by the total number of threads in the grid. This avoids requiring one GPU thread for every sample when `n_samples` is very large, improves occupancy control, and keeps memory access coalesced within each loop iteration because neighboring threads still read neighboring elements.

Question 3: Why is `(b - a) / n_samples` multiplication done only by thread 0?

Your answer: The block first reduces raw `y_i` values into one block sum. Applying `(b - a) / n_samples` once to the block sum avoids repeating the same scale operation for every sample and reduces arithmetic overhead. It also keeps the reduction focused on summing sample values, then converts the final block-level sum into an integral contribution only when writing the block result to global memory.

图 15: Lab5 Performance Scaling 分析截图

从 Test 4 到 Test 7, 样本数增加 100 倍, 运行时间增加约 $0.878/0.074 = 11.86$ 倍, 并非严格线性。小规模输入中内存分配、数据拷贝和 kernel launch 固定开销占比较高; 大规模输入则逐渐受内存带宽与 atomic contention 限制。

2.6 Lab6: K-Means

Assignment kernel 中每个点需要和所有 centroid 计算距离, 因此每个线程复杂度为 $O(k)$ 。当 $k = 100$ 时, 每个点都要进行 100 次距离比较。Update kernel 使用 shared memory atomics 先在 block 内聚合, 再写回 global memory, 以减少 global atomic 竞争。

$$dx^2 + dy^2 < 10^{-8}$$

是收敛判断条件。Double buffering 将旧 centroid 和新 centroid 分开存储, 既能安全比较收敛, 也避免覆盖当前迭代仍需使用的数据。

Exercise 1: Algorithm Analysis

Question 1: In the assignment kernel, what is the time complexity per thread if $k = 100$?

Your answer: $O(k)$, so for $k = 100$ each thread performs 100 distance comparisons. Each comparison computes a squared Euclidean distance and updates the current minimum, so the assignment kernel scales linearly with the number of clusters.

Question 2: Why does the recalculation kernel use shared memory atomics before global atomics?

Your answer: Shared memory atomics first aggregate points inside each block, which greatly reduces the number of slower global atomic operations and lowers contention on global memory. Instead of every point directly updating global centroid sums, each block produces partial sums locally and only merges k partial results to global memory.

Question 3: What happens if a centroid has no points assigned to it (empty cluster)?

Your answer: The implementation keeps that centroid at its previous position instead of dividing by zero or moving it to an invalid value. This preserves a valid centroid for the next iteration and avoids NaN values propagating through later distance calculations.

图 16: Lab6 K-Means 数据生成与编译

Exercise 3: Compute Intensity

For $N = 50,000$ points and $k = 50$ clusters:

Assignment kernel:

- Distance calculations per point: 50
- Total distance calculations: $50,000 * 50 = 2,500,000$

Update kernel:

- Shared memory atomics: $3 * 50,000 = 150,000$
- Global atomics (with 256 threads/block): $\text{ceil}(50,000 / 256) * 3 * 50 = 196 * 150 = 29,400$

Analysis: The assignment kernel performs far more arithmetic than the update kernel. The update kernel still matters because atomics can serialize when many points map to the same centroid, but the shared-memory reduction reduces global atomic pressure from one global update per point to one global merge per centroid per block.

图 17: Lab6 K-Means 测试样例运行结果

Exercise 5: Convergence Analysis

Question 1: What is the convergence threshold in this implementation?

Your answer: The squared centroid movement threshold is $1.0e-8$. A centroid is treated as converged when $dx^2 + dy^2 < 1.0e-8$. Since the comparison uses squared distance, the code avoids the extra cost of a square root while still measuring whether the centroid movement is very small.

Question 2: Why is it important to check convergence rather than always running `max_iterations`?

Your answer: Convergence checking avoids unnecessary iterations after the centroids have stabilized, reducing runtime and saving GPU work while producing the same final clusters. This is especially useful for large datasets because each extra iteration repeats both the $O(N * k)$ assignment step and the atomic update step.

Question 3: How does the double-buffering (swapping prev and new centroids) help?

Your answer: Double-buffering keeps the previous centroids and the newly computed centroids in separate arrays, so the program can safely compare old and new values and avoid overwriting data that is still needed during the current iteration. Swapping the pointers also avoids copying centroid arrays every iteration, making the iterative loop simpler and more efficient.

图 18: Lab6 K-Means 性能分析结果

截图显示程序能够完成数据生成、编译和测试。K-Means 的主导开销来自 assignment step, 因为每个点都需要遍历全部 centroid; 当输入规模、聚类中心数或迭代次数增加时, 运行时间会相应增长。

3 SVD GPU 实验

3.1 计算流程

SVD 框架主流程如下:

1. main.cpp 构造测试矩阵 A ;
2. to_bidiagonal 将 A 化为上二对角矩阵 B , 并累计正交矩阵 U, V ;
3. gkh_svd_from_bidiagonal 对 B 做 Golub-Kahan 迭代, 使其收敛为非负降序对角矩阵;
4. 主程序验证重构误差、正交误差、对角结构和奇异值排序。

框架保持 $A = UBV^T$ 不变, 最终 B 的对角线元素即为奇异值。

3.2 Householder 阶段的 GPU 化

bidiagonalization.cpp 中的 Householder 更新可写成 GEMV 与 GER:

$$w = v^T B_{\text{sub}}, \quad B_{\text{sub}} \leftarrow B_{\text{sub}} - \beta v w^T.$$

指导书要求关注外层循环 `for (int k = 0; k < n; ++k)` 中交替执行的左乘和右乘 Householder。每一步对 B, U, V 分别包含 GEMV 和 GER, 因此矩阵元素级更新天然适合 GPU 并行。

本实现选择手写 CUDA/HIP kernel 路线, 而不是直接调用 cuBLAS。原因是框架中的 `Matrix` 类采用行主序存储, 而 cuBLAS 默认列主序, 直接替换容易出现转置关系错误。手写 kernel 可以保持原有行主序布局, 并显式控制访存方式。

GPU Householder kernels

```
1 householder_dot_cols_kernel(...)
2 householder_dot_rows_kernel(...)
3 householder_left_update_kernel(...)
4 householder_rows_update_kernel(...)
```

其中 `dot_cols` 与 `dot_rows` 对应 GEMV, 分别计算 $v^T B_{\text{sub}}$ 和 $B_{\text{sub}} v$; `left_update` 与 `rows_update` 对应 GER, 每个线程负责更新矩阵中的一个元素。代码中还保留 `SVD_USE_CUDA`、`SVD_USE_HIP` 和默认 CPU/SIMD 回退路径。

3.3 GKH 阶段说明

本报告只把 `to_bidiagonal` 内部实现作为 GPU 化对象。服务器运行结果显示, GKH 迭代耗时显著大于上二对角化耗时, 所以完整 SVD 的总时间仍会受到后续 CPU 侧迭代限制。

bidiagonalization.cpp 中 SVD GPU kernel 完整代码块

```
1 #if defined(SVD_USE_HIP) && defined(__HIPCC__)
2 #include <hip/hip_runtime.h>
```

```

3  #define SVD_CUDA_ENABLED 1
4  #define cudaError_t hipError_t
5  #define cudaSuccess hipSuccess
6  #define cudaMalloc hipMalloc
7  #define cudaFree hipFree
8  #define cudaMemcpy hipMemcpy
9  #define cudaMemcpyHostToDevice hipMemcpyHostToDevice
10 #define cudaMemcpyDeviceToHost hipMemcpyDeviceToHost
11 #define cudaGetDeviceCount hipGetDeviceCount
12 #define cudaGetLastError hipGetLastError
13 #define cudaDeviceSynchronize hipDeviceSynchronize
14 #elif defined(SVD_USE_CUDA) && defined(__CUDACC__)
15 #include <cuda_runtime.h>
16 #define SVD_CUDA_ENABLED 1
17 #else
18 #define SVD_CUDA_ENABLED 0
19 #endif
20
21 #if SVD_CUDA_ENABLED
22 __global__ void householder_dot_cols_kernel(const double *matrix, const double *v,
23                                             double *out, int rows, int cols,
24                                             int row0, int col0,
25                                             int sub_rows, int sub_cols)
26 {
27     extern __shared__ double partial[];
28     const int col = blockIdx.x;
29     const int tid = threadIdx.x;
30
31     double sum = 0.0;
32     if (col < sub_cols)
33     {
34         for (int i = tid; i < sub_rows; i += blockDim.x)
35             sum += v[i] * matrix[(row0 + i) * cols + (col0 + col)];
36     }
37     partial[tid] = sum;
38     __syncthreads();
39
40     for (int stride = blockDim.x / 2; stride > 0; stride >>= 1)
41     {
42         if (tid < stride)
43             partial[tid] += partial[tid + stride];
44         __syncthreads();
45     }
46
47     if (tid == 0 && col < sub_cols)
48         out[col] = partial[0];
49 }
50
51 __global__ void householder_dot_rows_kernel(const double *matrix, const double *v,
52                                             double *out, int rows, int cols,
53                                             int row0, int col0,
54                                             int sub_rows, int sub_cols)
55 {
56     extern __shared__ double partial[];
57     const int row = blockIdx.x;
58     const int tid = threadIdx.x;
59
60     double sum = 0.0;
61     if (row < sub_rows)
62     {
63         for (int j = tid; j < sub_cols; j += blockDim.x)
64             sum += matrix[(row0 + row) * cols + (col0 + j)] * v[j];
65     }
66     partial[tid] = sum;
67     __syncthreads();
68

```

```

69     for (int stride = blockDim.x / 2; stride > 0; stride >>= 1)
70     {
71         if (tid < stride)
72             partial[tid] += partial[tid + stride];
73         __syncthreads();
74     }
75
76     if (tid == 0 && row < sub_rows)
77         out[row] = partial[0];
78 }
79
80 __global__ void householder_left_update_kernel(double *matrix, const double *v,
81                                               const double *w, double beta,
82                                               int rows, int cols, int row0,
83                                               int col0, int sub_rows,
84                                               int sub_cols)
85 {
86     const int col = blockIdx.x * blockDim.x + threadIdx.x;
87     const int row = blockIdx.y * blockDim.y + threadIdx.y;
88     if (row < sub_rows && col < sub_cols)
89         matrix[(row0 + row) * cols + (col0 + col)] -= beta * v[row] * w[col];
90 }
91
92 __global__ void householder_rows_update_kernel(double *matrix, const double *v,
93                                               const double *w, double beta,
94                                               int rows, int cols, int row0,
95                                               int col0, int sub_rows,
96                                               int sub_cols)
97 {
98     const int col = blockIdx.x * blockDim.x + threadIdx.x;
99     const int row = blockIdx.y * blockDim.y + threadIdx.y;
100    if (row < sub_rows && col < sub_cols)
101        matrix[(row0 + row) * cols + (col0 + col)] -= beta * w[row] * v[col];
102 }
103
104 static bool gpu_householder_left(Matrix &B, Matrix &U, int k,
105                                 const std::vector<double> &v, double beta)
106 {
107     if (!cuda_device_ready())
108         return false;
109
110     const int m = B.rows();
111     const int n = B.cols();
112     const int sub_rows = m - k;
113     const int sub_cols = n - k;
114     const int threads = 256;
115     const dim3 tile(16, 16);
116     const dim3 b_grid((sub_cols + tile.x - 1) / tile.x,
117                      (sub_rows + tile.y - 1) / tile.y);
118     const dim3 u_grid((sub_rows + tile.x - 1) / tile.x,
119                      (m + tile.y - 1) / tile.y);
120
121     DeviceBuffer d_B(B.size()), d_U(U.size()), d_v(v.size());
122     DeviceBuffer d_wB(sub_cols), d_wU(m);
123     if (!d_B.ok() || !d_U.ok() || !d_v.ok() || !d_wB.ok() || !d_wU.ok())
124         return false;
125
126     if (!copy_to_device(d_B, B.data()) || !copy_to_device(d_U, U.data()) ||
127         !copy_to_device(d_v, v.data()))
128         return false;
129
130     householder_dot_cols_kernel<<<sub_cols, threads, threads * sizeof(double)>>>(<
131         d_B.get(), d_v.get(), d_wB.get(), m, n, k, k, sub_rows, sub_cols);
132     if (!cuda_ok(cudaGetLastError()))
133         return false;
134 }

```

```

135     householder_left_update_kernel<<<b_grid, tile>>>(<
136         d_B.get(), d_v.get(), d_wB.get(), beta, m, n, k, k, sub_rows, sub_cols);
137     if (!cuda_ok(cudaGetLastError()))
138         return false;
139
140     householder_dot_rows_kernel<<<m, threads, threads * sizeof(double)>>>(<
141         d_U.get(), d_v.get(), d_wU.get(), m, m, 0, k, m, sub_rows);
142     if (!cuda_ok(cudaGetLastError()))
143         return false;
144
145     householder_rows_update_kernel<<<u_grid, tile>>>(<
146         d_U.get(), d_v.get(), d_wU.get(), beta, m, m, 0, k, m, sub_rows);
147     if (!cuda_ok(cudaGetLastError()) || !cuda_ok(cudaDeviceSynchronize()))
148         return false;
149
150     std::vector<double> host_B(B.size()), host_U(U.size());
151     if (!copy_to_host(host_B.data(), d_B) || !copy_to_host(host_U.data(), d_U))
152         return false;
153     std::copy(host_B.begin(), host_B.end(), B.data());
154     std::copy(host_U.begin(), host_U.end(), U.data());
155     return true;
156 }
157
158 static bool gpu_householder_right(Matrix &B, Matrix &V, int k,
159                                   const std::vector<double> &v, double beta)
160 {
161     if (!cuda_device_ready())
162         return false;
163
164     const int m = B.rows();
165     const int n = B.cols();
166     const int sub_rows = m - k;
167     const int sub_cols = n - k - 1;
168     const int threads = 256;
169     const dim3 tile(16, 16);
170     const dim3 b_grid((sub_cols + tile.x - 1) / tile.x,
171                       (sub_rows + tile.y - 1) / tile.y);
172     const dim3 v_grid((sub_cols + tile.x - 1) / tile.x,
173                       (n + tile.y - 1) / tile.y);
174
175     DeviceBuffer d_B(B.size()), d_V(V.size()), d_vec(v.size());
176     DeviceBuffer d_wB(sub_rows), d_wV(n);
177     if (!d_B.ok() || !d_V.ok() || !d_vec.ok() || !d_wB.ok() || !d_wV.ok())
178         return false;
179
180     if (!copy_to_device(d_B, B.data()) || !copy_to_device(d_V, V.data()) ||
181         !copy_to_device(d_vec, v.data()))
182         return false;
183
184     householder_dot_rows_kernel<<<sub_rows, threads, threads * sizeof(double)>>>(<
185         d_B.get(), d_vec.get(), d_wB.get(), m, n, k, k + 1, sub_rows, sub_cols);
186     if (!cuda_ok(cudaGetLastError()))
187         return false;
188
189     householder_rows_update_kernel<<<b_grid, tile>>>(<
190         d_B.get(), d_vec.get(), d_wB.get(), beta, m, n, k, k + 1, sub_rows, sub_cols);
191     if (!cuda_ok(cudaGetLastError()))
192         return false;
193
194     householder_dot_rows_kernel<<<n, threads, threads * sizeof(double)>>>(<
195         d_V.get(), d_vec.get(), d_wV.get(), n, n, 0, k + 1, n, sub_cols);
196     if (!cuda_ok(cudaGetLastError()))
197         return false;
198
199     householder_rows_update_kernel<<<v_grid, tile>>>(<
200         d_V.get(), d_vec.get(), d_wV.get(), beta, n, n, 0, k + 1, n, sub_cols);

```

```

201     if (!cuda_ok(cudaGetLastError()) || !cuda_ok(cudaDeviceSynchronize()))
202         return false;
203
204     std::vector<double> host_B(B.size()), host_V(V.size());
205     if (!copy_to_host(host_B.data(), d_B) || !copy_to_host(host_V.data(), d_V))
206         return false;
207     std::copy(host_B.begin(), host_B.end(), B.data());
208     std::copy(host_V.begin(), host_V.end(), V.data());
209     return true;
210 }
211 #endif

```

4 服务器运行与结果分析

4.1 服务器与本地运行结果

服务器侧在 master_ubss2 上使用种子 2412592 完成了 O2 优化运行,输出文件为 test_02_4cores.o:

Server O2 run

```
1 bash test.sh 5 1 4 -O2 -s 2412592
```

指标	服务器 O2 结果	说明
通过样例数	5 / 5	全部 PASS
1000 × 1000 上二对角化耗时	3456.54 ms	Householder 阶段
1000 × 1000 GKH 耗时	23967.5 ms	后续迭代阶段
总上二对角化耗时	3456.61 ms	5 个测试合计
总 GKH 耗时	23968.4 ms	5 个测试合计

表 6: 服务器 O2 运行结果

本地 Windows 环境也使用同一份 GPU/code 代码重新编译并运行 CPU baseline:

Local CPU run

```

1 g++ -std=c++17 -O2 -pthread main.cpp bidiagonalization.cpp gkh.cpp -o main_local.exe
2 ./main_local.exe 2412592

```

本地运行同样得到 5 / 5 PASS, 其中 1000 × 1000 样例上二对角化耗时约为 2824.13 ms, GKH 迭代耗时约为 5234.98 ms; 5 个测试合计上二对角化耗时为 2824.13 ms, GKH 迭代耗时为 5238.27 ms。本地 nvidia-smi 显卡为 NVIDIA GeForce RTX 4060 Laptop GPU, 驱动版本为 610.47, CUDA UMD 版本为 13.3。

4.2 本地 NVIDIA GPU 测试

补充使用当前 Python 环境中的 torch 2.11.0+cu128 运行 CUDA smoke test。测试脚本为 GPU/code/local_nvidia_gpu_smoke.py, 输出保存为 local_nvidia_gpu_smoke.json:

Local NVIDIA GPU test

```
1 python local_nvidia_gpu_smoke.py
```

项目	结果	说明
GPU	NVIDIA GeForce RTX 4060 Laptop GPU	8 GB 级显存
Compute Capability	8.9	Ada Lovelace 移动端 GPU
PyTorch CUDA	12.8	torch 2.11.0+cu128
4096 × 4096 CPU matmul	285.372 ms	CPU 对照
4096 × 4096 GPU matmul	23.599 ms	CUDA Event 计时
矩阵乘加速比	12.09×	CPU / GPU
Householder-like 更新	2.523 ms	GEMV + GER 型微基准

表 7: 本地 NVIDIA GPU test 结果

该测试证明本地 RTX 4060 可以通过 CUDA 路径正常执行大规模矩阵运算,并且对 GEMV/GER 风格的 Householder-like 更新也能给出毫秒级执行时间。

4.3 本地 nvcc CUDA C++ SVD 运行

在完成 CUDA Toolkit 13.3 与 Visual Studio 2022 Build Tools 安装后,本地已能使用 nvcc 编译 bidiagonalization.cpp 中真正嵌入 SVD 程序的 CUDA kernel。由于 main.cpp 与 gkh.cpp 是普通 C++ 代码,而 bidiagonalization.cpp 内含 __global__ kernel,本次采用分步编译:普通 C++ 文件由 MSVC 编译, GPU kernel 文件由 nvcc -x cu 编译,最后再链接为 svd_cuda.exe。

Local nvcc CUDA C++ build

```

1 cl /std:c++17 /O2 /EHsc /DNOMINMAX /utf-8 /c main.cpp gkh.cpp
2 nvcc -x cu -std=c++17 -O2 -DSVD_USE_CUDA -DSVD_PROFILE_GPU ^
3   -Xcompiler "/utf-8/EHsc/DNOMINMAX" ^
4   -c bidiagonalization.cpp -o bidiagonalization_cuda.obj
5 nvcc main.obj gkh.obj bidiagonalization_cuda.obj -o svd_cuda.exe
6 svd_cuda.exe 2412592 > local_nvcc_cuda_run.txt 2>&1

```

项目	本地 CUDA C++ 结果
GPU	NVIDIA GeForce RTX 4060 Laptop GPU
显存	8188 MiB
驱动版本	610.47
CUDA Toolkit	13.3, nvcc 13.3.33
MSVC Build Tools	Visual Studio 2022 BuildTools 17.14.35
随机种子	2412592
通过样例数	5 / 5

表 8: 本地 nvcc 编译链与 CUDA C++ 运行环境

指标	本地 CPU O2	本地 CUDA C++
1000 × 1000 上二对角化耗时	2824.13 ms	17315.3 ms
1000 × 1000 GKH 耗时	5234.98 ms	5983.64 ms
总上二对角化耗时	2824.13 ms	17730.1 ms
总 GKH 耗时	5238.27 ms	5987.52 ms
通过样例数	5 / 5	5 / 5

表 9: 本地 CPU O2 与 nvcc CUDA C++ 端到端结果对比

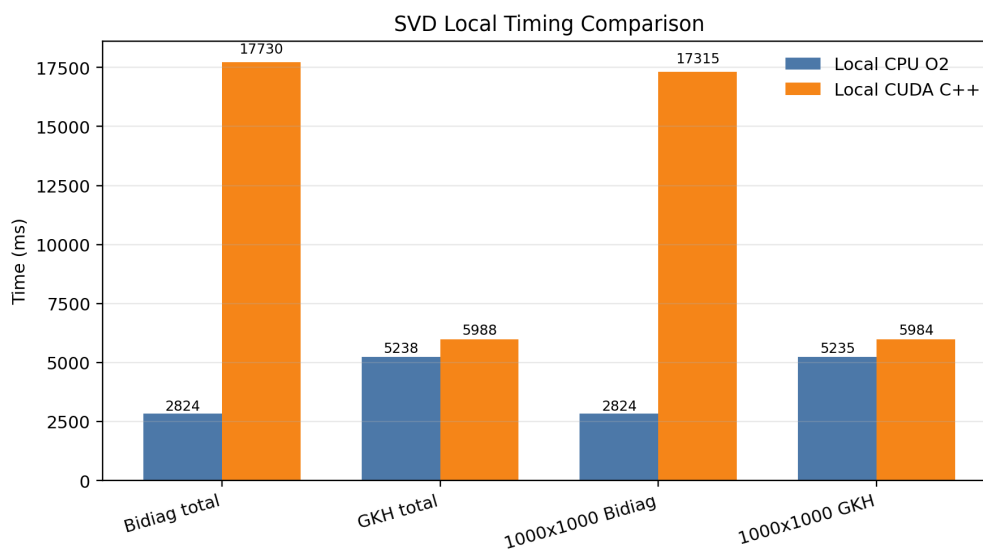


图 19: 本地 CPU O2 与 CUDA C++ SVD 耗时对比图

GPU profile 指标	数值	说明
H2D 拷贝时间	3825.486 ms	Host 到 Device
D2H 拷贝时间	3097.562 ms	Device 到 Host
kernel 同步时间	431.515 ms	GPU kernel 执行与同步
H2D 数据量	30479.484 MB	累计传输量
D2H 数据量	30471.852 MB	累计传输量
左 Householder 调用	1026 次	gpu_householder_left
右 Householder 调用	1019 次	gpu_householder_right

表 10: SVD_GPU_PROFILE 输出的本地 CUDA 运行统计

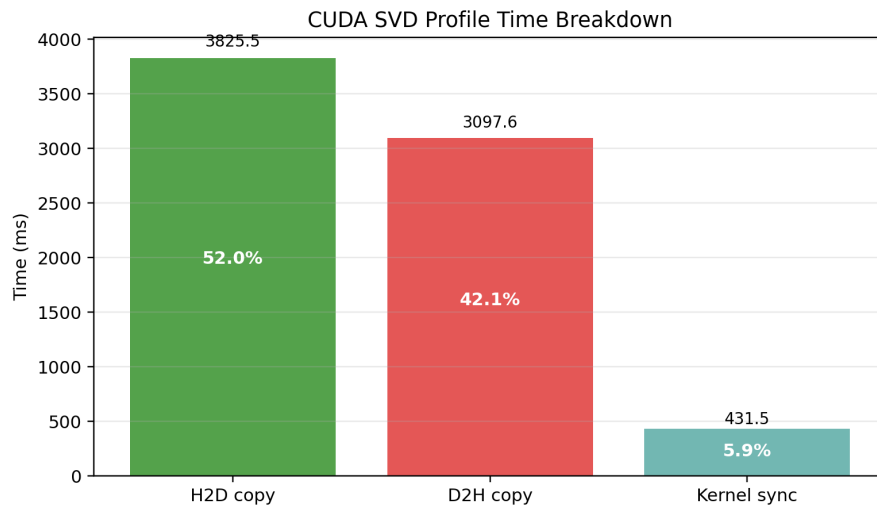


图 20: 本地 CUDA SVD profile 时间分解图

从 profile 可见, kernel 本身累计同步时间为 431.515 ms, 低于本地 CPU O2 的上二对角化总时间; 但当前实现为了兼容原有 **Matrix** 接口, 每次左/右 Householder 都重新分配设备缓冲并进行整矩阵 H2D/D2H 拷贝, 累计传输量超过 60 GB。最终 CUDA C++ 端到端上二对角化耗时反而达到 17730.1 ms, 约为本地 CPU O2 的 $17730.1/2824.13 \approx 6.28$ 倍。这说明手写 GPU kernel 的计算部分已经能够工作, 但数据驻留和传输策略是当前版本的主要瓶颈。

4.4 VTune 与 AMD uProf profiling

使用 VTune 对 `main_local.exe` 进行 hotspots 采样:

热点函数	CPU Time	占比
<code>apply_right_cols_range</code>	3.539 s	37.4%
<code>main_local.exe</code> 未细分符号	1.626 s	17.2%
<code>simd_add_scaled</code>	1.610 s	17.0%
<code>simd_dot</code>	1.004 s	10.6%
<code>cleanup_bidiagonal</code>	0.921 s	9.7%
其他	0.755 s	8.0%

表 11: 本地 VTune hotspots 采样结果

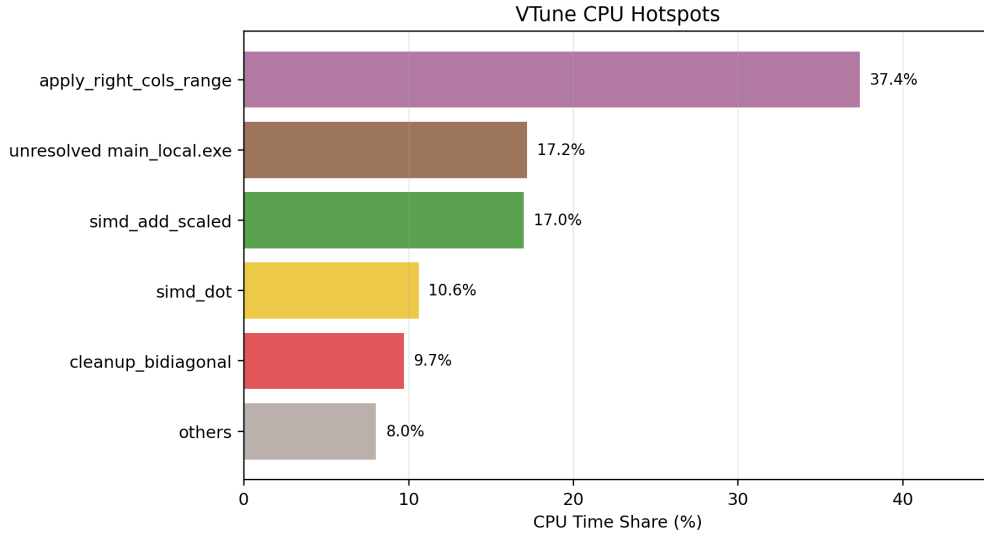


图 21: VTune CPU hotspots 占比图

VTune 报告显示总 elapsed time 为 10.019 s, CPU time 为 9.455 s, 采样方式为 user-mode sampling and tracing。热点主要集中在 `bidiagonalization.cpp` 的 CPU Householder 路径, 尤其是右侧列更新、SIMD 向量加法、SIMD 点积和双对角清理。这说明 CPU baseline 的性能瓶颈落在上二对角化内部的 GEMV/GER 型更新循环中, 也正对应本实验选择 GPU 化的核心动机。

4.5 结果分析

服务器 O2 结果中, 1000×1000 样例的 GKH 耗时约为上二对角化耗时的

$$\frac{23967.5}{3456.54} \approx 6.93$$

倍; 本地 CPU 对照运行中, 该比例约为

$$\frac{5234.98}{2824.13} \approx 1.85$$

倍。两组数据的绝对时间受硬件、编译器和运行环境影响较大, 但共同说明完整 SVD 并不只由上二对角化决定, 后续 GKH 迭代仍会限制总加速。因此即使 `to_bidiagonal` 内部 GEMV/GER 已经接入 GPU 路径, 完整程序的端到端加速也可能被 CPU 侧 GKH 阶段抵消。

此外, 本地 `nvcc` 运行进一步验证了这一点: `SVD_GPU_PROFILE` 中 kernel 同步时间只有 431.515 ms, 但 H2D/D2H 拷贝时间合计约为 6923.048 ms, 且每次 Householder 都伴随设备内存分配、整矩阵复制和同步。若矩阵不能长期驻留在 GPU 显存中, 通信与调度开销会抵消 kernel 并行带来的收益。说明后续优化方向应优先考虑持久化设备端矩阵、批量执行多个 Householder 更新以及减少全矩阵回拷。

5 总结

本报告整理了 AMD HIP 六个 notebook 的关键代码填空、截图结果和性能分析, 并针对 SVD 选题说明了 Householder 上二对角化阶段的 GPU 化实现。代码层面, `bidiagonalization.cpp` 已经提供手写 CUDA/HIP kernel 路径, 覆盖 GEMV/GER 型更新。服务器 O2 运行在学号种子 2412592 下通过全部 5 个测试样例, 说明当前实现保持了 SVD 分解正确性。本地进一步完

成 CUDA Toolkit 13.3 与 Visual Studio 2022 Build Tools 编译链配置, 使用 `nvcc` 成功构建并运行 `svd_cuda.exe`, CUDA C++ 路径同样 5 / 5 PASS。profiling 显示 kernel 累计同步时间为 431.515 ms, 但 H2D/D2H 拷贝时间合计约 6923.048 ms, 导致端到端上二对角化慢于本地 CPU O2。该结果说明 GPU kernel 的计算正确性已经得到验证, 而性能瓶颈主要来自当前实现的数据搬运和设备内存生命周期管理。

6 SVD 选题 AI 问答记录



图 22: SVD AI 问答截图 1

当前 Matrix 类是行主序存储，而 cuBLAS 默认按列主序理解矩阵。我选择手写 CUDA/HIP kernel，而不是直接替换为 cuBLAS 调用，这个选择有什么优缺点？

Qwen3.7-Max 14:12:48

在行主序 (Row-Major) 存储下选择手写 Kernel 而非直接调用列主序 (Column-Major) 的 cuBLAS，是一个在工程便利性与极致性能之间的典型权衡。以下是具体的优缺点分析：

✅ 优点

1. 避免转置开销 (最核心优势)

- cuBLAS 原生假设列主序。若强行用于行主序矩阵，通常需要将 $M \times N$ 的矩阵视为 $N \times M$ 的列主序矩阵来调用 (即逻辑转置)，或者在调用前后进行物理转置。
- 对于 Bidiagonalization 这种迭代算法，每一步 Householder 变换都涉及 GEMV/GER。如果每步都依赖 cuBLAS 的逻辑转置技巧或物理转置，累积的额外开销可能抵消库函数本身的优化收益。手写 Kernel 可以直接按行主序索引 `A[row * ld + col]`，零额外开销。

2. Kernel Fusion 的天然支持

- 如上一轮分析所述，GEMV ($w = v^T B$) 和 GER ($B \leftarrow B - \beta v v^T$) 融合是本题的关键优化点。
- cuBLAS 是黑盒 API，无法将两个操作融合为一个 Kernel。手写 Kernel 可以在 Shared Memory 中完成 w 的计算后立即用于更新 B ，消除中间向量 w 的全局内存读写，这在带宽受限的 GEMV 操作中收益巨大。

3. 精确控制数据布局与访存模式

- 行主序下，行的访问是连续的，列的访问是跨步的。手写 Kernel 可以针对性地设计 Thread Block 形状 (如 `dim3(32, 8)`) 和循环方向，确保 Global Memory Coalescing。
- 可以将 Householder 向量 v 放入 `__constant__ memory` 而 cuBLAS 的通用接口不一定能利用这一优化。

4. 教学与调试价值

向千问提问

✳ 任务助理

🔍 深度思考

🔧 深度研究

📄 代码

🖼 图像

☰ 更多

🌐

@

↑

图 23: SVD AI 问答截图 2

服务器 O2 结果中，1000×1000 样例上二对角化约为 3456.54 ms，GKH 约为 23967.5 ms。即使上二对角化能 GPU 加速，总时间可能仍不明显下降。这个现象应该怎样分析？

Qwen3.7-Max

这是一个非常经典的**Amdahl 定律（阿姆达尔定律）**陷阱，也是并行计算实验中极具教学价值的现象。二对角化仅占总耗时的 **12.6%** ($3456 / (3456 + 23967)$)，这意味着即使你将二对角化加速到“瞬间完成” (0ms)，总时间的理论上限也仅能提升约 14%。

要深入分析这一现象并找到突破口，建议从以下四个维度展开：

1. 量化加速瓶颈：Amdahl 定律分析

首先用数据明确 GPU 加速的理论天花板，避免盲目优化低收益部分。

表格					
阶段	CPU 耗时 (ms)	占比	GPU 理想加速比	加速后该阶段耗时	总时间节省上限
二对角化	3,456	12.6%	∞ (瞬时)	0 ms	12.6%
GKH 迭代	23,967	87.4%	1× (未加速)	23,967 ms	0%
总计	27,423	100%	-	23,967 ms	12.6%

- 结论：在当前算法流程下，GPU 加速二对角化的边际收益极低。除非 GKH 也被并行化，否则总时间不会显著下降。

2. 诊断 GKH 为何成为瓶颈

GKH (Golub-Kahan-Householder) 迭代占 87%，需要分析其内部构成：

向千问提问

✳ 任务助理

🧐 深度思考

🔍 深度研究

📄 代码

🖼 图像

☰ 更多

🌐

🔗

📤

图 24: SVD AI 问答截图 3

7 截图与材料清单

编号	已插入截图	内容
1	q1,q12,q17	Lab1 设备检测、block size、sub-wavefront
2	q21,q25,q2c	Lab2 转置 benchmark 与访存分析
3	q31,q35,q3s	Lab3 histogram benchmark 与 speedup
4	q41,q43,q44	Lab4 reduction 与 occupancy sweep
5	q51,q54,q56	Lab5 Monte Carlo scaling
6	q61,q63,q65	Lab6 K-Means 编译、测试与性能
7	test_02_4cores.o、 local_cpu_02.txt、 local_nvidia_gpu_smoke.json、nvcc CUDA C++ 运行与 profile、本地 local_nvcc_cuda_run.txt、 local_nvcc_build.log、 vtune_summary.txt	服务器 O2 结果、本地 CPU 对照、本地 NVIDIA GPU smoke test、本地 VTune hotspots
8	svd_timing_compare.png、 svd_cuda_profile_time.png、 svd_vtune_hotspots.png	数据分析图表：端到端耗时对比、CUDA profile 分解、CPU hotspots 占比
9	ai1.png, ai2.png, ai3.png	SVD AI 问答过程截图

表 12: 截图与材料清单

8 AMD Notebook 补充截图附录

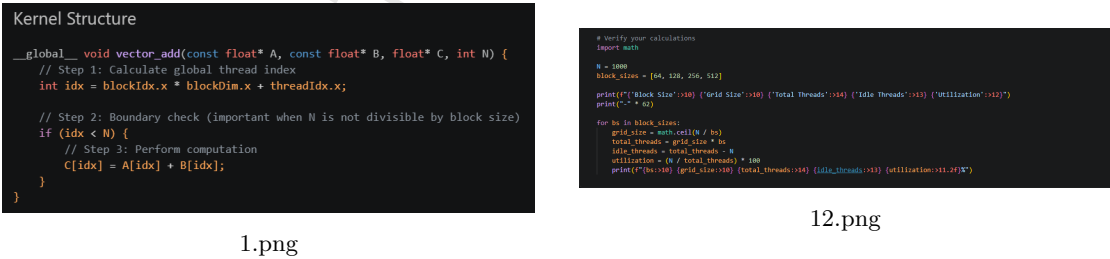


图 25: AMD HIP notebook 补充截图：1.png 与 12.png


```

__global__ void transpose_tiled(const float* input, float* output,
                               int rows, int cols) {
    __shared__ float tile[BLOCK_SIZE][BLOCK_SIZE + 1]; // +1 to avoid bank conflicts

    int x = blockIdx.x * BLOCK_SIZE + threadIdx.x;
    int y = blockIdx.y * BLOCK_SIZE + threadIdx.y;

    // Read coalesced (row-major)
    if (x < cols && y < rows)
        tile[threadIdx.y][threadIdx.x] = input[y * cols + x];
    __syncthreads();

    // Calculate output position (transposed block)
    x = blockIdx.y * BLOCK_SIZE + threadIdx.x;
    y = blockIdx.x * BLOCK_SIZE + threadIdx.y;

    // Write coalesced (row-major in output)
    if (x < rows && y < cols)
        output[y * rows + x] = tile[threadIdx.x][threadIdx.y];
}

```

123.png

Algorithm

Each block is responsible for counting **one specific bin**:

```

__global__ void kernel(const int* input, int* histogram, int N, int num_bins) {
    __shared__ int sdata[1024];

    int tid = threadIdx.x;
    int whichnumbins = blockIdx.x; // Each block handles one bin

    sdata[tid] = 0;

    // Each thread scans entire array looking for matches
    for (int i = tid; i < N; i += blockDim.x) {
        if (input[i] == whichnumbins)
            sdata[tid]++;
    }
    __syncthreads();

    // Reduction to sum counts
    for (int i = blockDim.x / 2; i > 0; i >>= 1) {
        if (tid < i) sdata[tid] += sdata[tid + i];
        __syncthreads();
    }

    if (tid == 0) histogram[whichnumbins] = sdata[0];
}

```

131.png

图 29: AMD HIP notebook 补充截图: 123.png 与 131.png

```

Algorithm
__global__ void kernel(const int* input, int* histogram, int N, int num_bins) {
    __shared__ int sdata[1024]; // Local histogram per block

    int idx = threadIdx.x + blockIdx.x * blockDim.x;
    int totalthread = blockDim.x * gridDim.x;

    // Step 1: Initialize shared memory histogram
    for (int i = threadIdx.x; i < num_bins; i += blockDim.x) {
        sdata[i] = 0;
    }
    __syncthreads();

    // Step 2: Accumulate into shared memory (local atomics)
    for (int i = idx; i < N; i += totalthread) {
        atomicAdd(&sdata[input[i]], 1);
    }
    __syncthreads();

    // Step 3: Merge local histograms into global (global atomics)
    for (int i = threadIdx.x; i < num_bins; i += blockDim.x) {
        atomicAdd(&histogram[i], sdata[i]);
    }
}

```

132.png

```

import subprocess, re, statistics

# Multi-trial benchmark - parse REAL compute time from stderr (KERNEL_MS)
TRIALS = 5
KERN_RE = re.compile(r"KERNEL_MS\s+([0-9.e+]*)\s+")

impls = [
    ("Serial CPU", "../exe_serial"),
    ("Naive GPU", "../exe_naive"),
    ("Optimized GPU", "../exe_optimized"),
]

testcases = [
    ("Test 2 (medium)", "testcases/2.in"),
    ("Test 4 (large)", "testcases/4.in"),
]

def run_once(exe, path):
    p = subprocess.run([exe, path], capture_output=True, text=True)
    m = KERN_RE.search(p.stderr)
    if not m:
        raise RuntimeError(f"No KERNEL_MS from {exe} {path}: {p.stderr[:200]}")
    return float(m.group(1))

results = {} # results[testcase][impl] - list of ms
for tc_name, tc_path in testcases:
    results[tc_name] = []
    for impl_name, exe in impls:
        times = [run_once(exe, tc_path) for _ in range(TRIALS)]
        results[tc_name][impl_name] = times
        m, s = statistics.mean(times), statistics.stdev(times) if len(times) > 1 else 0.0
        print(f"({tc_name:10}) {impl_name:10} mean={m:8.4f} ms std={s:8.4f} ms")
    print()

```

133.png

图 30: AMD HIP notebook 补充截图: 132.png 与 133.png

```

# Left: grouped bar chart per test case (log scale)
s = np.arange(len(tc_names))
w = 0.27

for i, impl in enumerate(impl_names):
    means = [statistics.mean(results[tc][impl]) for tc in tc_names]
    stds = [statistics.stdev(results[tc][impl]) if len(results[tc][impl]) > 1 else 0.0
            for tc in tc_names]
    bars = ax1.bar(s * (1 - 3) * w, means, w, yerr=stds, capsize=4,
                  label=impl, color=colors[i], edgecolor='black')
    for b, m in zip(bars, means):
        ax1.text(b.get_x() + b.get_width()/2, m, f"({m:.2f})",
                 ha='center', va='bottom', fontsize=9)
    ax1.set_ylabel("Compute time (ms, log)")
    ax1.set_xlabel("Test Case")
    ax1.set_title("Histogram: Real Compute Time (TRIALS) trials")
    ax1.legend(); ax1.grid(axis='y', alpha=0.3, which='both')

# Right: Speedup of optimized over naive and serial
speedup_vs_serial = [statistics.mean(results[tc]["Serial CPU"]) /
                    statistics.mean(results[tc]["Optimized GPU"]) for tc in tc_names]
speedup_vs_naive = [statistics.mean(results[tc]["Naive GPU"]) /
                   statistics.mean(results[tc]["Optimized GPU"]) for tc in tc_names]

w2 = 0.35
ax2.bar(s - w2/2, speedup_vs_serial, w2, label="Optimized / Serial",
        color="#55a677", edgecolor='black')
ax2.bar(s + w2/2, speedup_vs_naive, w2, label="Optimized / Naive",
        color="#c44e52", edgecolor='black')
for i, (s1, s2) in enumerate(zip(speedup_vs_serial, speedup_vs_naive)):
    ax2.text(i - w2/2, s1, f"({s1:.1f})", ha='center', va='bottom', fontsize=9)
    ax2.text(i + w2/2, s2, f"({s2:.1f})", ha='center', va='bottom', fontsize=9)
ax2.set_xlabel("Speedup (x)")
ax2.set_ylabel("Speedup (x)")
ax2.set_title("Speedup of Optimized GPU over (true compute, excluding I/O)")
ax2.axhline(1, color='black', linewidth=2)
ax2.legend(); ax2.grid(axis='y', alpha=0.3, which='both')

```

134.png

```

import numpy as np, os, subprocess, re, statistics
import matplotlib.pyplot as plt

# Atomic contention experiment - use REAL kernel time so the differences
# between distributions are not drowned in I/O.
N, NPM_BINS = 10_000_000, 64
KERN_RE = re.compile(r"KERNEL_MS\s+([0-9.e+]*)\s+")

def write_case(path, data):
    with open(path, "w") as f:
        f.write(f"({N} NPM_BINS)\n")
        f.write(" ".join(map(str, data.tolist())))

rng = np.random.default_rng(0)
dists = {
    "Uniform": rng.integers(0, NPM_BINS, N),
    "Gaussian": np.clip(rng.normal(NPM_BINS/2, NPM_BINS/8, N).astype(int), 0, NPM_BINS-1),
    "All-same(0)": np.zeros(N, dtype=int),
}

paths = {n: f"testcases/dist/{n}.in" for n in dists}
for n, d in dists.items():
    if not os.path.exists(paths[n]):
        write_case(paths[n], d)

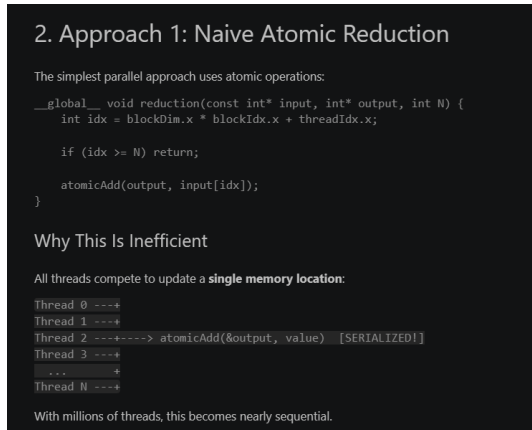
TRIALS = 5
dist_times = {}
for name, p in paths.items():
    ts = []
    for _ in range(TRIALS):
        p = subprocess.run(["../exe_optimized", p], capture_output=True, text=True)
        m = KERN_RE.search(p.stderr)
        if not m:
            raise RuntimeError(f"No KERNEL_MS for {p}")
        ts.append(float(m.group(1)))
    dist_time[name] = ts
    print(f"({name:10}) mean={statistics.mean(ts):8.4f} ms std={statistics.stdev(ts):8.4f} ms")

means = [statistics.mean(dist_times[n]) for n in dists]
stds = [statistics.stdev(dist_times[n]) for n in dists]

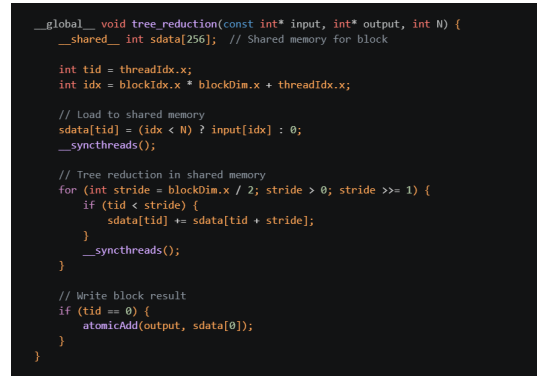
```

135.png

图 31: AMD HIP notebook 补充截图: 134.png 与 135.png

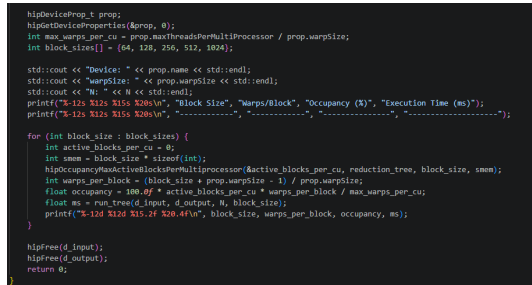


141.png

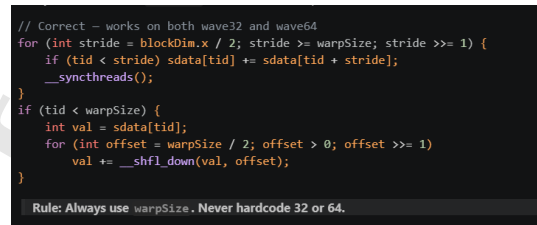


143.png

图 32: AMD HIP notebook 补充截图: 141.png 与 143.png

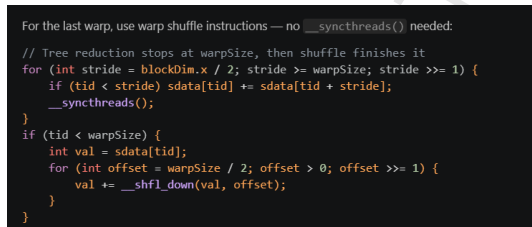


144.png

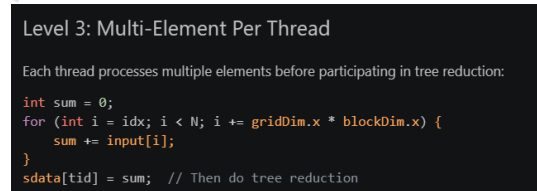


145.png

图 33: AMD HIP notebook 补充截图: 144.png 与 145.png

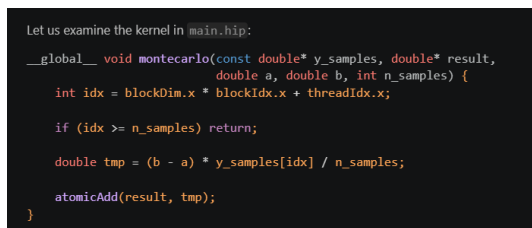


146.png

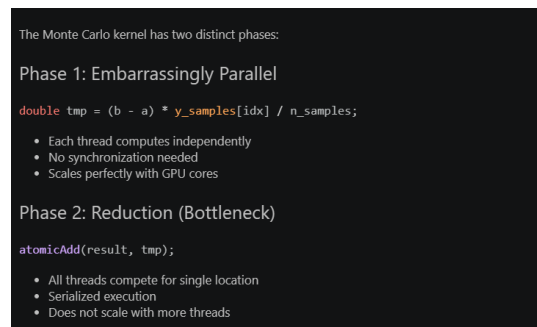


147.png

图 34: AMD HIP notebook 补充截图: 146.png 与 147.png



151.png



154.png

图 35: AMD HIP notebook 补充截图: 151.png 与 154.png

```

__global__ void montecarlo_optimized(const double* y_samples, double* result,
                                   double a, double b, int n_samples) {
    __shared__ double sdata[256];

    int tid = threadIdx.x;
    int idx = blockIdx.x * blockDim.x + threadIdx.x;

    // Phase 1: Each thread accumulates multiple elements
    double local_sum = 0.0;
    for (int i = idx; i < n_samples; i += blockDim.x * gridDim.x) {
        local_sum += y_samples[i];
    }
    sdata[tid] = local_sum;
    __syncthreads();

    // Phase 2: Tree reduction in shared memory
    for (int s = blockDim.x / 2; s > 0; s >>= 1) {
        if (tid < s) {
            sdata[tid] += sdata[tid + s];
        }
        __syncthreads();
    }

    // Only one atomic per block
    if (tid == 0) {
        atomicAdd(result, (b - a) * sdata[0] / n_samples);
    }
}

```

l55.png

```

__global__ void find_nearest_centroids_kernel(
    const float* data_x, const float* data_y, int* labels,
    const float* centroids_x, const float* centroids_y,
    int sample_size, int k)
{
    int point_id = blockIdx.x * blockDim.x + threadIdx.x;
    if (point_id < sample_size) {
        int nearest = 0;
        float min_dist = INFINITY;
        for (int j = 0; j < k; ++j) {
            float dx = centroids_x[j] - data_x[point_id];
            float dy = centroids_y[j] - data_y[point_id];
            float dist = dx*dx + dy*dy;
            if (dist < min_dist) {
                min_dist = dist;
                nearest = j;
            }
        }
        labels[point_id] = nearest;
    }
}

```

l61.png

图 36: AMD HIP notebook 补充截图: l55.png 与 l61.png

```

Kernel 2: Recalculate Centroids

__global__ void recalculate_centroid_kernel(
    const float* data_x, const float* data_y, const int* labels,
    int* centroid_point_nums, float* new_centroids_x, float* new_centroids_y,
    int sample_size, int k)
{
    __shared__ float new_centroids_x_shared[100];
    __shared__ float new_centroids_y_shared[100];
    __shared__ int centroid_point_nums_shared[100];

    // Initialize shared memory
    if (threadIdx.x < k) {
        new_centroids_x_shared[threadIdx.x] = 0;
        new_centroids_y_shared[threadIdx.x] = 0;
        centroid_point_nums_shared[threadIdx.x] = 0;
    }
    __syncthreads();

    // Accumulate to shared memory (local atomics)
    int point_id = blockIdx.x * blockDim.x + threadIdx.x;
    if (point_id < sample_size) {
        int cid = labels[point_id];
        atomicAdd(new_centroids_x_shared + cid, data_x[point_id]);
        atomicAdd(new_centroids_y_shared + cid, data_y[point_id]);
        atomicAdd(centroid_point_nums_shared + cid, 1);
    }
    __syncthreads();

    // Merge to global memory (one atomic per block)
    if (threadIdx.x < k) {
        atomicAdd(new_centroids_x + threadIdx.x, new_centroids_x_shared[threadIdx.x]);
        atomicAdd(new_centroids_y + threadIdx.x, new_centroids_y_shared[threadIdx.x]);
        atomicAdd(centroid_point_nums + threadIdx.x, centroid_point_nums_shared[threadIdx.x]);
    }
}

```

l62.png

```

Kernel 3: Finalize and Check Convergence

__global__ void finalize_recalculation_and_compare_kernel(
    const int* centroid_point_nums,
    float* new_centroids_x, float* new_centroids_y,
    const float* prev_centroids_x, const float* prev_centroids_y,
    int k, int* num_converged_centroids)
{
    int idx = threadIdx.x;

    if (idx == 0) *num_converged_centroids = 0;
    __syncthreads();

    if (idx < k) {
        int count = centroid_point_nums[idx];
        if (count > 0) {
            new_centroids_x[idx] /= count;
            new_centroids_y[idx] /= count;
        } else {
            // Keep previous centroid if no points assigned
            new_centroids_x[idx] = prev_centroids_x[idx];
            new_centroids_y[idx] = prev_centroids_y[idx];
        }

        // Check convergence
        float dx = new_centroids_x[idx] - prev_centroids_x[idx];
        float dy = new_centroids_y[idx] - prev_centroids_y[idx];
        if (dx*dx + dy*dy < 1.0e-8f) {
            atomicAdd(num_converged_centroids, 1);
        }
    }
}

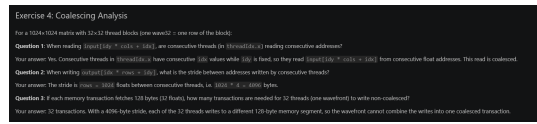
```

l63.png

图 37: AMD HIP notebook 补充截图: l62.png 与 l63.png

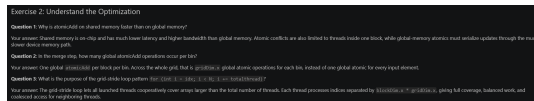


q23.png



q24.png

图 42: AMD HIP notebook 补充截图: q23.png 与 q24.png



q32.png

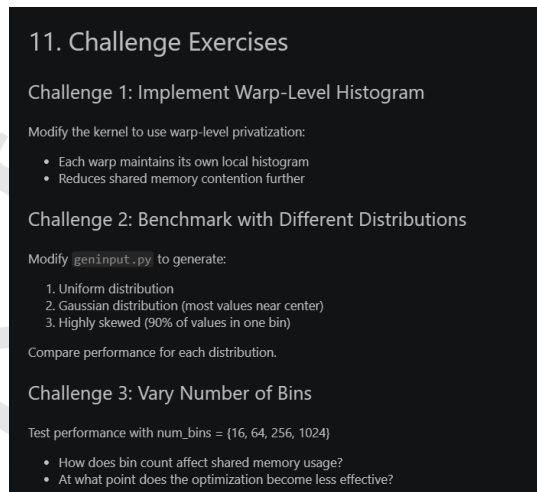


q33.png

图 43: AMD HIP notebook 补充截图: q32.png 与 q33.png



q34.png

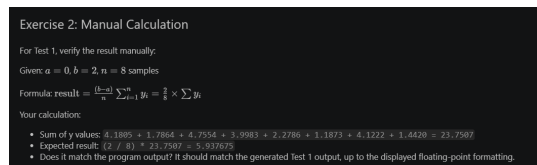


q3c.png

图 44: AMD HIP notebook 补充截图: q34.png 与 q3c.png

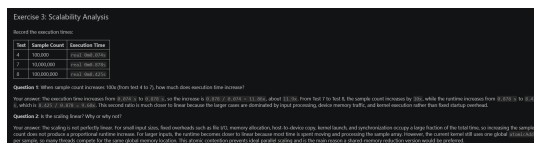


q42.png

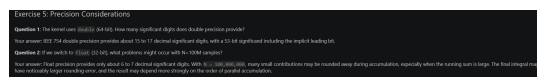


q52.png

图 45: AMD HIP notebook 补充截图: q42.png 与 q52.png



q53.png



q55.png

图 46: AMD HIP notebook 补充截图: q53.png 与 q55.png



q62.png



q64.png

图 47: AMD HIP notebook 补充截图: q62.png 与 q64.png

参考文献

[1] 并行程序设计课程组. 2026 并行程序设计 Lab5 GPU 编程.

[2] Golub, G. H. and Van Loan, C. F. Matrix Computations.